



EFREI Paris

Formation *Big Data Framework*

Hubble Lakehouse

Rapport

Pipeline Big Data pour l'analyse des observations
du télescope spatial Hubble

Auteurs

Valentin Massonnière
Kevin Heugas

27 avril 2026

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Objectifs du projet	4
1.3	Livrables attendus	4
2	Périmètre fonctionnel	5
2.1	Profils utilisateurs	5
2.2	Hors-périmètre	5
3	Sources de données	6
3.1	Source 1 — MAST	6
3.2	Source 2 — NOAA SWPC	6
3.3	Choix des sources	6
4	Architecture	7
4.1	Vue d'ensemble	7
4.2	Couches Médaillon	7
5	Spécifications fonctionnelles	8
5.1	Ingestion	8
5.2	Transformations par couche	8
5.3	Datamarts	8
5.4	Restitution	8
6	Spécifications techniques	9
6.1	Volumétrie	9
6.2	Orchestration	9
6.3	Qualité et reprise	9
6.4	Bonus — déploiement conteneurisé	9
7	Modèle de données	10
7.1	Datamart Physicien	10
7.2	Datamart Chimiste	10
7.3	Datamart Biologiste	10
7.4	Datamart Ingénieur	10
8	Répartition des tâches	11
8.1	Valentin	11
8.2	Kevin	11
8.3	Tâches communes	11
9	Détail des livrables	12
9.1	Dépôt GitHub	12
9.2	Rapport	12
10	Déploiement de l'infrastructure	13
10.1	Vue d'ensemble	13
10.2	Conteneurisation	13
10.3	Manifestes Kubernetes	13
10.4	Authentification Airflow	13
10.5	Exposition HTTPS	14
10.6	Pipeline CI/CD	14

10.7 Prochaines étapes	14
10.8 Difficultés	14

Note méthodologique. Ce rapport a d’abord été rédigé en brouillon par nous-mêmes. Une IA nous a aidé à nous corriger les fautes et reformuler certaines phrases, sans modifier les justifications ni ajouter de contenu. L’objectif est de gagner du temps de rédaction et de limiter les fautes.

1 Introduction

1.1 Contexte

Lancé en 1990 par la NASA et l'ESA, le télescope spatial Hubble (HST) constitue depuis plus de 30 ans l'un des instruments scientifiques le plus puissant de l'histoire de l'astronomie. Ses observations couvrent un large spectre, de l'ultraviolet au proche infrarouge, et alimentent des champs de recherche aussi variés que la cosmologie, l'étude des exoplanètes ou la physique stellaire.

L'ensemble des observations produites par Hubble est mis à disposition du public via deux archives principales : le *Mikulski Archive for Space Telescopes* (MAST) opéré par le STScI, et l'*ESA Hubble Science Archive* opéré par l'ESAC.

1.2 Objectifs du projet

Le projet vise à concevoir, implémenter et documenter une **architecture Lakehouse complète** suivant la logique *Médaille* (couches Bronze, Silver, Gold), capable de :

- **Ingérer** de manière fiable deux sources hétérogènes : un historique massif d'observations Hubble (batch) et un flux continu de météo spatiale (streaming) ;
- **Stocker** la donnée brute dans un système de fichiers distribué (HDFS) sans transformation préalable, garantissant rejouabilité et auditabilité ;
- **Nettoyer, typer et structurer** la donnée dans une couche intermédiaire (Hive / Parquet) servant de socle commun à l'ensemble des cas d'usage ;
- **Modéliser et matérialiser** dans une base relationnelle (PostgreSQL) quatre datamarts métier répondant à des problématiques scientifiques et techniques concrètes ;
- **Tracer** chaque étape du pipeline par des logs consultables depuis le dashboard Airflow.

1.3 Livrables attendus

Les livrables attendus sont :

- un **rapport écrit** décrivant l'architecture, les choix techniques, les jeux de données mobilisés et les résultats obtenus ;
- un **dépôt GitHub** regroupant l'ensemble du code source du pipeline.

2 Périmètre fonctionnel

2.1 Profils utilisateurs

Le projet répond à quatre problématiques posées par quatre métiers scientifiques.

Physicien. Quelles régions du ciel ont été sur-observées par Hubble, et quels sont les angles morts de la recherche ?

Chimiste. Quelles cibles ont été observées avec des filtres à bande étroite correspondant à des raies d'émission spécifiques ($H\alpha$, OIII, SII...)?

Biologiste. Quelles observations ont été acquises pendant une éruption solaire ou un événement de météo spatiale, et risquent d'être corrompues par les radiations ?

Ingénieur spatial. Comment évoluent les temps de calibration des instruments Hubble au fil du temps ?

2.2 Hors-périmètre

Le projet ne couvre pas :

- l'analyse du contenu des images FITS ; seules les métadonnées sont exploitées ;
- la prédiction (apprentissage automatique) ; les datamarts servent au reporting et à l'exploration ;

3 Sources de données

3.1 Source 1 — MAST

MAST est l'archive publique des observations Hubble, accessible via le [portail web](#) du STScI. Les métadonnées sont exportées au format CSV. Le fichier utilisé contient environ 500 000 lignes, décrites par le schéma CAOM (*Common Archive Observation Model*).

3.2 Source 2 — NOAA SWPC

NOAA SWPC publie en quasi-temps réel les indicateurs de météo spatiale : vent solaire, flux X-ray, indice planétaire Kp, éruptions solaires. Les données sont disponibles au format JSON sur des endpoints HTTP (services.swpc.noaa.gov/json/), la récupération se fait en mode streaming.

3.3 Choix des sources

MAST fournit un historique statique traité en batch. NOAA SWPC fournit un flux temps réel traité en streaming.

4 Architecture

4.1 Vue d'ensemble

L'architecture suit le patron Médaille, organisé en trois couches successives : Bronze, Silver et Gold. Les données circulent des sources jusqu'aux outils de restitution en passant par chacune de ces couches. La figure 1 présente le pipeline complet.

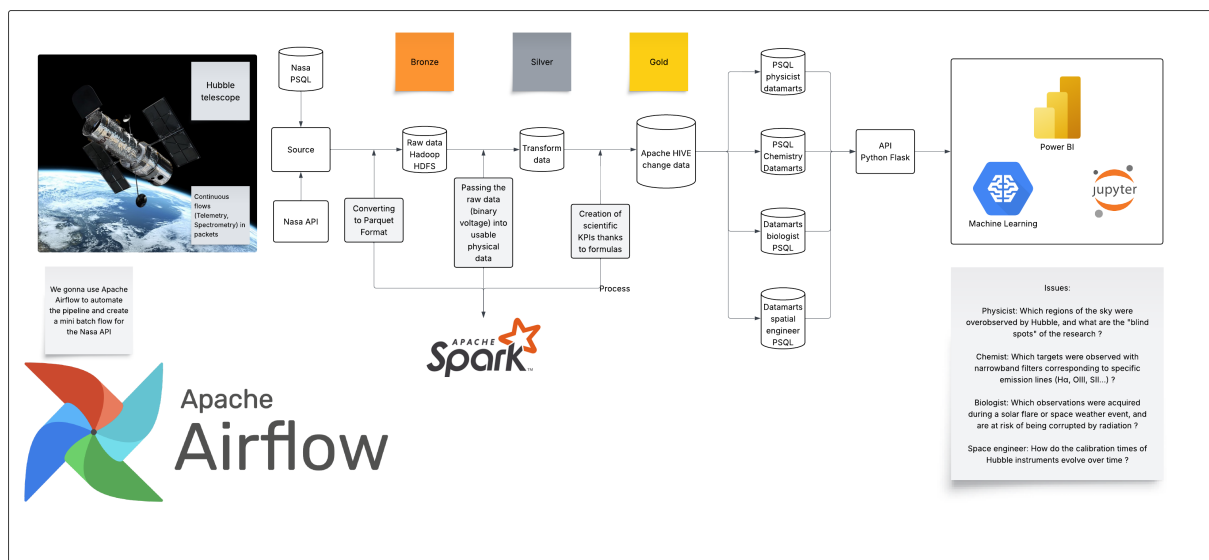


FIGURE 1 – Architecture Lakehouse du projet.

4.2 Couches Médaille

Bronze. Stockage sur HDFS au format Parquet. Les exports MAST (CSV) et les flux NOAA SWPC (JSON) sont convertis en Parquet sans transformation métier. La couche est immuable et sert de référence en cas de retraitement.

Silver. Nettoyage et structuration via Apache Spark. Les données sont typées, parsées (filtres, dates MJD), converties au format Parquet, et exposées via Apache Hive.

Gold. Agrégation et calcul des indicateurs métier. Quatre tables PostgreSQL alimentent les quatre datamarts, une par profil utilisateur. Une API REST développée en Python avec Flask expose ces tables, et l'accès est protégé par jeton JWT.

5 Spécifications fonctionnelles

5.1 Ingestion

MAST. Le fichier CSV est téléchargé depuis le portail MAST en mode batch. Un job Airflow déclenche un script Python qui dépose le fichier dans HDFS Bronze, après conversion au format Parquet.

NOAA SWPC. Les endpoints JSON sont interrogés en mini-batch toutes les minutes. Un job Spark Structured Streaming lit les réponses HTTP et les écrit dans HDFS Bronze au format Parquet.

5.2 Transformations par couche

Bronze → Silver. Spark lit les fichiers Parquet bruts et applique le typage des colonnes, le parsing des champs structurés (filtres, polygones, dates MJD vers UTC...) et la suppression des colonnes redondantes. Les tables Silver sont stockées au format Parquet et déclarées dans Hive.

Silver → Gold. Spark SQL agrège les tables Silver pour produire les indicateurs métier. Les résultats sont chargés dans PostgreSQL via JDBC.

5.3 Datamarts

Quatre datamarts PostgreSQL sont produits, un par profil utilisateur.

Physicien. Savoir quelles régions du ciel Hubble a beaucoup observées, et lesquelles sont restées dans l'ombre.

Chimiste. Retrouver les images prises à travers des filtres capables de révéler certains gaz dans l'espace, comme l'hydrogène, l'oxygène ou le soufre.

Biologiste. Repérer les observations prises pendant une éruption solaire, qui risquent d'être abîmées par les radiations.

Ingénieur spatial. Suivre l'usure des instruments de Hubble en regardant comment leur temps de calibration évolue au fil des années.

5.4 Restitution

Les quatre tables PostgreSQL sont exposées par une API REST écrite en Python avec Flask, accessible à l'adresse <https://efrei-bigdata-api.valentin-massonniere.ch>. Chaque datamart correspond à un endpoint dédié, qui retourne les enregistrements au format JSON. L'accès est protégé par jeton JWT : chaque requête doit présenter un token valide dans son en-tête **Authorization**. Les outils de visualisation et les scripts d'analyse consomment ces endpoints, sans accès direct à la base.

6 Spécifications techniques

6.1 Volumétrie

L'export MAST représente environ 500 000 observations, soit quelques centaines de méga-octets en Parquet. Le flux NOAA SWPC ajoute quelques kilo-octets par minute.

6.2 Orchestration

Apache Airflow pilote l'ensemble du pipeline batch via un DAG quotidien. Le flux NOAA SWPC est porté par un job Spark Structured Streaming permanent.

6.3 Qualité et reprise

Chaque couche est idempotente : un retraitement ne crée pas de doublons. Les enregistrements rejetés (champs manquants, valeurs incohérentes) sont conservés dans une table de rejet par couche.

6.4 Bonus — déploiement conteneurisé

En complément du périmètre commun, Valentin déploie le pipeline sur un VPS personnel. Chaque composant est conteneurisé avec Docker et orchestré par Kubernetes.

Les logs des jobs sont centralisés et consultables depuis le dashboard Airflow, accessible à l'adresse <https://efrei-bigdata-airflow.valentin-massonniere.ch> (utilisateur `invit`, mot de passe `inviter123`). Pour des raisons de sécurité du VPS, ce compte dispose d'un accès en lecture seule et ne peut pas déclencher de DAG manuellement.

7 Modèle de données

Chaque profil utilisateur est servi par une table PostgreSQL dédiée, alimentée par un job Spark SQL depuis la couche Silver. Les schémas des sources sont documentés ci-dessous :

7.1 Datamart Physicien

Quelles régions du ciel ont été sur-observées par Hubble, et quels sont les angles morts de la recherche ?

Colonne	Type	Description
ra_bin	float	Position est-ouest dans le ciel (sorte de longitude céleste).
dec_bin	float	Position nord-sud dans le ciel (sorte de latitude céleste).
nb_obs	int	Nombre de fois où cette zone du ciel a été photographiée.
total_exptime	float	Temps total passé à observer cette zone, en secondes.

7.2 Datamart Chimiste

Quelles cibles ont été observées avec des filtres à bande étroite correspondant à des raies d'émission spécifiques (H α , OIII, SII...)?

Colonne	Type	Description
target_name	string	Nom de l'objet observé (étoile, nébuleuse, galaxie...).
filter	string	Filtre coloré placé devant l'instrument pendant la prise.
emission_line	string	Gaz que ce filtre permet de mettre en évidence (hydrogène, oxygène, soufre...).
nb_obs	int	Nombre d'observations correspondantes.

7.3 Datamart Biologiste

Quelles observations ont été acquises pendant une éruption solaire ou un événement de météo spatiale, et risquent d'être corrompues par les radiations ?

Colonne	Type	Description
obs_id	string	Référence unique de l'observation.
t_start	timestamp	Date et heure du début de la prise.
t_end	timestamp	Date et heure de fin de la prise.
flare_class	string	Intensité de l'éruption solaire qui a eu lieu pendant la prise.
risk_score	int	Niveau de risque que l'image soit abîmée par les radiations.

7.4 Datamart Ingénieur

Comment évoluent les temps de calibration des instruments Hubble au fil du temps ?

Colonne	Type	Description
instrument	string	Nom de l'instrument de Hubble (caméra, spectrographe...).
year_month	date	Mois auquel la mesure correspond.
avg_calib_time	float	Temps moyen passé à régler l'instrument ce mois-là, en secondes.
trend_12m	float	Évolution sur les 12 derniers mois ; une valeur positive signifie que l'instrument met de plus en plus de temps à se régler.

8 Répartition des tâches

Le projet est mené en binôme. La répartition s'appuie sur les appétences techniques de chacun.

8.1 Valentin

- Mise en place et maintenance du dépôt GitHub.
- Déploiement de l'infrastructure (HDFS, Spark, Hive, PostgreSQL, Airflow) sur un VPS personnel via Docker et Kubernetes (bonus).
- Ingestion du flux NOAA SWPC via Spark Structured Streaming.
- Transformations Silver et Gold liées aux événements de météo spatiale.
- Datamarts Biologiste et Ingénieur.
- Coordination générale du projet.

8.2 Kevin

- Ingestion batch de l'export MAST via Python et Airflow.
- Transformations Silver des observations Hubble (typage, parsing des filtres, conversion MJD vers UTC...).
- Modélisation des tables Hive de la couche Silver.
- Datamarts Physicien et Chimiste.

8.3 Tâches communes

- Choix techniques structurants (formats, schémas Gold, conventions de nommage).
- Rédaction du rapport.

9 Détail des livrables

9.1 Dépôt GitHub

Le dépôt est hébergé à l'adresse github.com/TheVall/efrei-projet-big-data. Il regroupe l'ensemble des artefacts du projet, organisés comme ceci :

- `ingestion/` : scripts Python d'ingestion MAST et du flux NOAA SWPC ;
- `spark/` : jobs Spark des transformations Bronze → Silver → Gold ;
- `sql/` : scripts SQL de création des datamarts PostgreSQL ;
- `dags/` : DAG Airflow d'orchestration du pipeline ;
- `infra/` : Dockerfiles et manifestes Kubernetes pour le déploiement bonus sur VPS ;
- `docs/` : documentation technique complémentaire ;
- `README.md` : guide de démarrage et instructions d'exécution.

9.2 Rapport

Ce document intègre le cahier des charges et sera complété au fil du projet par :

- l'architecture finale réellement implémentée ;
- les choix techniques et leurs justifications ;
- les résultats obtenus pour chaque datamart, avec exemples ;
- les difficultés rencontrées et les pistes d'amélioration.

10 Déploiement de l'infrastructure

Cette section décrit l'état actuel du déploiement bonus sur VPS personnel. Le périmètre couvert à ce stade se limite à Airflow et à l'API de restitution. Les composants Hadoop, Hive, Spark et PostgreSQL applicatifs seront ajoutés plus tard.

10.1 Vue d'ensemble

Le pipeline est déployé sur un cluster Kubernetes hébergé sur un VPS personnel, dans le namespace `efrei-big-data`. Trois composants tournent à ce stade :

- **PostgreSQL 16** : sert de backend de métadonnées pour Airflow et hébergera plus tard le metastore Hive et les datamarts Gold.
- **Airflow 3.0** : orchestrateur, en mode *standalone*, executor `LocalExecutor`.
- **API Flask** : point d'entrée de restitution, qui exposera les datamarts une fois ces derniers produits.

10.2 Conteneurisation

Chaque composant est conteneurisé. Deux images sont construites depuis le dépôt :

- `infra/Dockerfile.airflow` : image Airflow avec un *entrypoint* qui génère le fichier de mots de passe `simple_auth_manager_passwords.json.generated` à partir des variables d'environnement, puis lance `airflow standalone` ;
- `infra/Dockerfile.api` : image Python qui exécute l'API Flask via Gunicorn.

PostgreSQL utilise l'image officielle `postgres:16`, sans build custom.

10.3 Manifestes Kubernetes

Le fichier `infra/k8s.yaml` regroupe l'ensemble des ressources : namespace, `PersistentVolumeClaim` pour les données Postgres, `ConfigMap` d'initialisation de la base, trois `Deployment`, trois `Service`, deux `Ingress`, et un `Secret` `efrei-secrets` créé dynamiquement à chaque déploiement par la pipeline CI/CD.

L'utilisateur `airflow` de la base PostgreSQL est créé au premier démarrage par un script SQL injecté via `ConfigMap`.

10.4 Authentification Airflow

Airflow est configuré avec son *simple auth manager*. Deux comptes sont définis dans la variable `AIRFLOW__CORE__SIMPLE_AUTH_MANAGER_USERS` :

- `admin` (rôle `admin`) : accès complet, utilisé pour la maintenance ;
- `invit` (rôle `viewer`) : accès en lecture seule pour le jury, sans possibilité de déclencher de DAG.

Les mots de passe sont fournis par variables d'environnement, elles-mêmes injectées depuis le `Secret` Kubernetes.

10.5 Exposition HTTPS

Deux **Ingress** routent le trafic externe vers les services internes :

- `efrei-bigdata-airflow.valentin-massonniere.ch` → Airflow ;
- `efrei-bigdata-api.valentin-massonniere.ch` → API Flask.

Les certificats TLS sont émis automatiquement par *cert-manager* avec l'issuier **letsencrypt-prod**, via une simple annotation sur chaque **Ingress**. Aucun renouvellement manuel n'est nécessaire.

10.6 Pipeline CI/CD

Le dépôt est miroiré sur GitHub et GitLab. La pipeline GitLab (`.gitlab-ci.yml`) déclenche à chaque push sur `main` :

1. un job *build-airflow* qui construit l'image Airflow et la pousse sur le registry GitLab ;
2. un job *build-api* qui fait de même pour l'image API ;
3. un job *deploy* qui se connecte au cluster (via **kubeconfig** stocké en variable CI/CD), recrée les **Secret** Kubernetes à partir des variables protégées, applique `infra/k8s.yaml` et redémarre les déploiements.

Les variables sensibles (mots de passe, kubeconfig, identifiants registry) sont stockées comme variables CI/CD protégées GitLab et ne quittent jamais le runner.

10.7 Prochaines étapes

Les prochaines technologies seront ajoutées dans les itérations suivantes : HDFS et YARN (single-node), Hive (Metastore et HiveServer2), client Spark intégré à l'image Airflow, et exposition des UI HDFS et YARN sur deux **Ingress** dédiés pour les captures du livrable.

10.8 Difficultés

Le déploiement de Hadoop, Hive et Spark sur Kubernetes représente la principale difficulté technique pour la suite du projet. Valentin, qui prend en charge la partie infrastructure, n'a jamais réalisé ce type de déploiement et la conteneurisation de ces écosystèmes avec Kubernetes. La construction de ces fichiers s'appuiera sur la documentation et sur l'usage de l'IA pour comprendre les choix de configuration et produire une première version des manifestes, qui sera ensuite vérifiée et adaptée.

Hubble Lakehouse

Pipeline Big Data pour l'analyse des observations du télescope spatial Hubble

Signatures

Valentin Massonnière

Efrei Paris

M1 Data Engineering & AI — DE3

Kevin Heugas

Efrei Paris

M1 Data Engineering & AI — DE3

Fait à Paris, le 27 avril 2026.